

Fall 2026 CPSC 323-09 Project 1

Instructions:

The scope of this project is to understand how lexical analysis works. This is directly **applicable to your exams**, so again, **know how it works**. You will build and make use of **Deterministic Finite State Automata** to define items needed for programming. Although the lecture focuses on the **academic** side, the project is designed to give you a **practical** hands on approach to the subject of compilers. You are allowed to write your project in C/C++/Java/Python, but you ARE NOT allowed to use **Yacc, Bison, or any other similar items** that assist in the creation of compilers. This includes **NO Regular Expression libraries**.

Project 1 consists of several questions to be answered in this document and three programs to be submitted/uploaded online on Canvas. **Name each program as stated**; the extension may depend on the programming language of your choice. **I will deduct 5 points each time you name it differently as stated**. Maximum 80 points.

For Question 13, one may use the sample file F26_Q13_infile.txt

For Question 14, one may use the sample file F26_Q14_samples.txt

Both above files can be found on the project assignment page.

Project Part 1 | True/False, Regular Expression Fundamentals [6 points]

Instructions: For part 1, answer all the following True/False questions. For each question, give a small justification for your answer.

1. State the correctness of the statement below, false or true:
ababab is in $a^*(ba)^*$

False. $ababab$
 $a^*(ba)^*$. Let $x = 2 \rightarrow aababab$

2. State the correctness of the statement below, false or true:
Language generated by the regular expression $(ab)^*(ba)^*$ is the same as the language generated by the regular expression $(ab|ba)^*$

False. $(ab)^*(ba)^*$. Let $x = 2 \rightarrow abababba$
 $(ab|ba)^*$. Let $x = 2 \rightarrow abbaabba$

3. State the correctness of the statement below, false or true:
ababa is in $(ab)^*(ba)^*$

False. $(ab)^*(ba)^*$. Let $x = 2 \rightarrow abababba + ababa$

4. State the correctness of the statement below, false or true:
Language generated by the regular expression $(ab)^*(ba)^*$ is the same as the language generated by the regular expression $(abba)^*$

False. $(ab)^*(ba)^*$. Let $x = 2 \rightarrow abababba$
 $(abba)^*$. Let $x = 2 \rightarrow abbaabba$

5. State the correctness of the statement below, false or true:
Language generated by the regular expression $(ab|\epsilon)^*$ is the same as the language generated by the regular expression $(ab)^*$.

True. $(ab|\epsilon)^*$. Let $x = 3 \rightarrow ab\epsilon ab \rightarrow abab$
 $(ab)^*$. Let $x = 2 \rightarrow abab$

6. State the correctness of the statement below, false or true:
Language generated by the regular expression $(a^*|b^*)(b|\epsilon)$ is the same as the language generated by the regular expression $a^*|b^*$.

False. $(a^*|b^*)(b|\epsilon) \rightarrow bb$
 $a^*|b^* \rightarrow a$

Project Part 2 | Regular Expressions & FSM Implementation

[28 points]

Instructions: For part 2-5, answer all the essay questions below. For some questions, you must create a program. Upload that program along with the rest of your files when you submit.

7. (3 points)

Given the language $L = (a^* | b) b^* a$ over the alphabet $\{ a, b \}$, describe an FSM for L (can be NFSM or a DFSM). You may either draw the FSM or describe it formally, but the states Q , the start state q_0 , the accepting states F , and the transition function δ must be clearly specified.



8. (6 points) $M = (\Sigma = \{a, b\}, Q = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}, q_0 = 1, F = \{9\})$

Given the language $L = (a^* | b) a^*$ over the alphabet $\{a, b\}$, describe the DFSM that accepts L . If you wrote a NFSM, then convert it to a DFSM. You may either draw the DFSM or describe it formally, but the states Q , the start state q_0 , the accepting states F , and the transition function δ must be clearly specified.

[illegible]

9. (16 points)

Given the language $L = (a^* | b) b^* a$ over the alphabet $\{a, b\}$, use the transition function δ as part of a program that determines whether a given string is accepted or not by the DFSM. Write a program to read from the user a string of characters ending with \$, for the DFSM of the language L to determine whether the string is accepted or rejected by L .

Example:

input = aaababa\$ output=NO

input = aaa\$ output = YES.

Save it as Prog1 and upload it on Canvas. If you have multiple files, please zip the file into one file called Prog1 and upload it.

10. (3 points)

Given the language $L = (a^* | b) b^* a$ over the alphabet $\{a, b\}$, show or upload a snapshot image of the lines of code in the program implementing the transition table.

Project Part 3 | Language Recognition, RE and DFSM [10 Points]

11. (4 points)

Let L be the set of all strings in $\{a, b\}^*$ that contain at least two occurrences of the substring aba. The occurrences can be overlapping partially, but not totally.

Hint: The shortest string in L has the length 5; for example, the substring "ababa" contains two occurrences of the substring aba.

Write a regular expression that describes that language, and justify why your regular expression is correct.

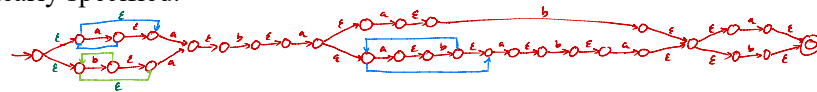
$$\begin{aligned}
 t &= abcbab \rightarrow (a|b)^* (aba (a|b)^* ab) (a|b)^* \\
 &\rightarrow a \underset{a \quad ba}{ba} \left[\begin{aligned} &(a|b)^* (abcbab | aba (a|b)^* aba) (a|b)^* \\ &\boxed{(a|b)^* (aba (ba | (a|b)^* aba) (a|b)^*} \end{aligned} \right.
 \end{aligned}$$

12. (6 points)

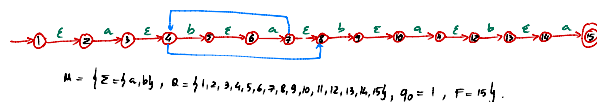
Let L be the set of all strings in $\{a, b\}^*$ that contain at least two occurrences of the substring aba . The occurrences can be overlapping partially, but not totally.

Hint: The shortest string in L has the length 5; for example, the substring "ababa" contains two occurrences of the substring aba.

Describe a DFSM over the alphabet $\Sigma = \{a, b\}$ that accepts the language L . You may either draw the DFSM or describe it formally, but the states Q , the start state q_0 , the accepting states F , and the transition function δ must be clearly specified.



Convert RE $\rightarrow$ NFSM $\rightarrow$ DFSM



Project Part 4 | Lexeme Preprocessing [12 Points]

13. (12 points)

Given an input text file “infile.txt”, write a program that does the following:

(i) filters out, i.e. removes, all comment lines (lines that begin with //), all empty or blank lines, and all extra white spaces between lexemes from the file “file.txt” and

(iii) ensures that there is only one space before and one space after each lexeme, by placing a space if needed before or after a lexeme, and

(iii) stores the resulting characters into a file called “outfile.txt”.

Please note that the cin>> is one lexeme. Save it as Prog2 and upload it on Canvas.

For example, for the sample input file “infile.txt” below, the output file “outfile.txt” should look like below:

Sample input “infile.txt” file	Correct output “outfile.txt”
<pre>//declare variables int var1, var2, var3 ,var4 ; //initialize variables var1 =13; vare2 = 55 ; var3 = 17 ; cin>> var4 ; //compute and display their total var1= var2 * var3 *var4; //compute product var4 = var4/var1; //compute new value for var4</pre>	<pre>int var1 , var2 , var3 , var4 ; var1 = 13 ; vare2 = 55 ; var3 = 17 ; cin>> var4 ; var1 = var2 * var3 * var4 ; var4 = var4 / var1 ;</pre>

Project Part 5 | Lexical Analysis/Token Classification

[24 Points]

14. (24 points)

Write a program to use the following arrays:

```
char reservedWords[4][10]={ "cin>>", "for", "int", "cout<<" };
char operator[6][2]={ "+", "-", "*", "/", "++", "--" }
char special[7][3]={ ">", "=", ",", "(", ")", ">=", ";" };
```

and do the following:

- (i) it reads from the user a single line (up to 255 characters) at a time,
- (ii) it will attempt to split the line into lexemes, if possible,
- (iii) it will determine and display on the output each lexeme and whether the lexeme is a reserved word, an arithmetic operator, a special symbol, a number, an identifier or an invalid lexeme. A number is an unsigned integer or a signed integer. An identifier starts with a letter or `_` followed by any number of letters, `_`, and digits, and excludes any reserved word; the reserved words are the one listed in the array `reservedWords[]`.

For each lexeme check whether it is a reserved word. If not, check whether it is a special symbol. If not, check whether it is a number. If not, check whether it is an identifier. If none of these, display "invalid". The program must be case insensitive. Save it as Prog3 and upload it on Canvas.

Sample I/O execution (On next page):

Enter a statement:

cin>> i+1 ; For (int 2j = 1 ; j <= i ; j = ++ j + 1) ; cout<< j /2i;	
cin>>	reserved word
i	identifier
+	operator
1	number
;	special symbol
for	reserved word
(	special symbol
int	reserved word
2j	invalid
=	special symbol
1	number
;	special symbol
j	identifier
<=	invalid
i	identifier
;	special symbol
j	identifier
=	special symbol
++	operator
j	identifier
+	operator
1	number
)	special symbol
cout<<	reserved word
j	identifier
/	operator
2i	invalid

CONTINUE (y/n)? y

Enter a statement:

int -5 = 2j ; for (j > 10) cout>> myTime ;	
int	reserved word
-5	number
=	special symbol
2j	invalid
;	special symbol
for	reserved word
(	special symbol
j	identifier
>	operator
10	number
)	special symbol
cout>>	invalid
myTime	identifier
;	special symbol

CONTINUE (y/n)? n